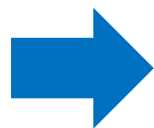


学习内容

CONTENTS



1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类与降维

5

决策树

6

强化学习



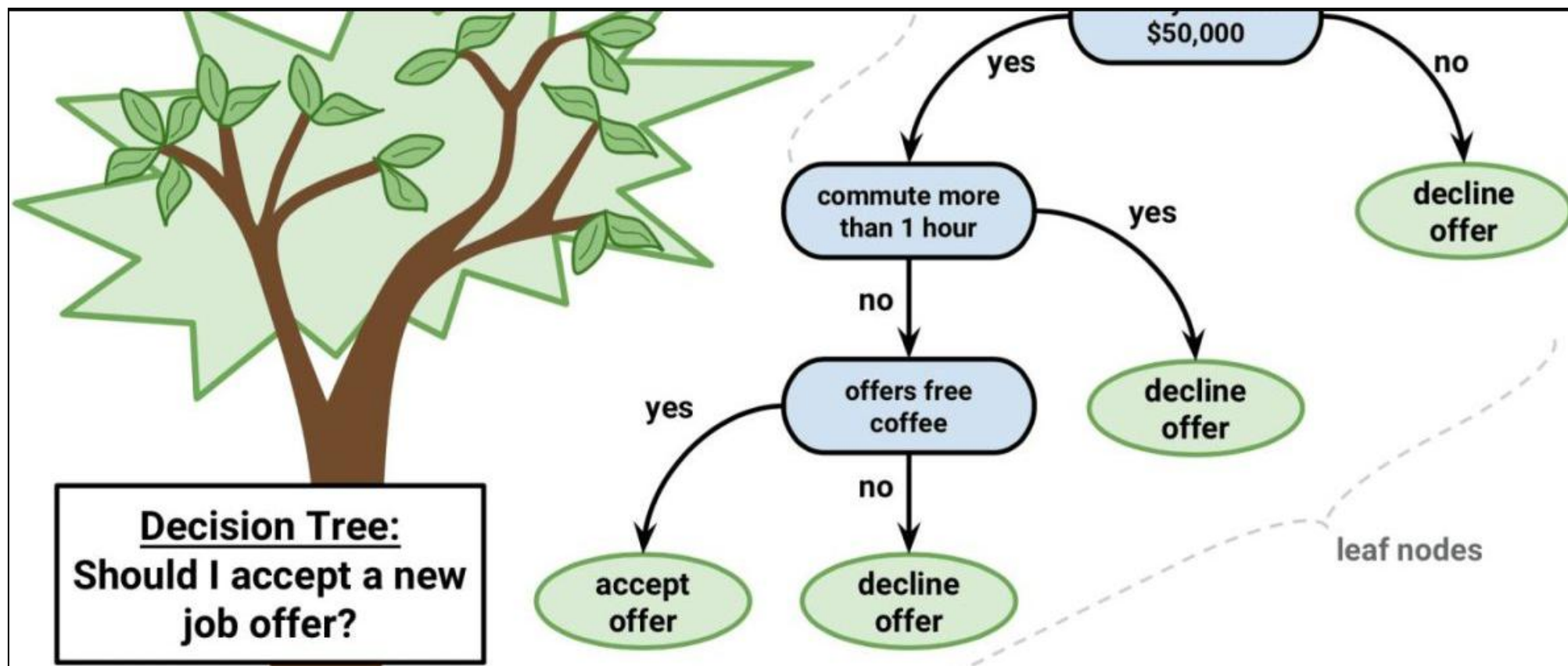
主要内容：

1. 掌握决策树算法的基本原理及基本应用
2. 理解决策树构建算法及其优缺点对比

重点：

掌握决策树的构建算法：ID3算法

决策树 (decision tree) 是一个树结构, 每个内部节点表示一个特征属性, 每个分支代表这个特征属性在某个值域上的输出, 每个树叶节点存放一个类别。





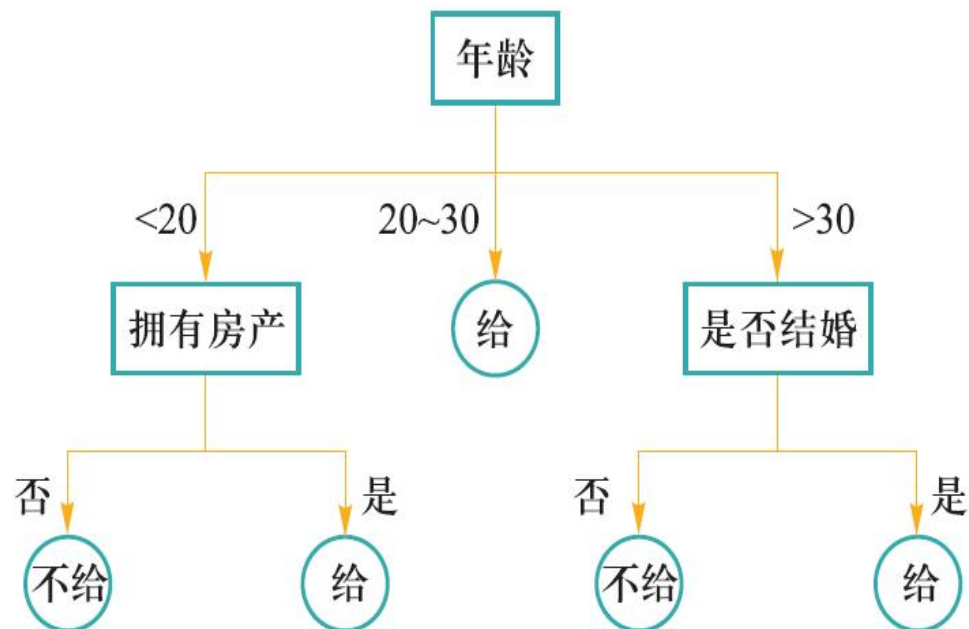


是否制作精良	口碑好	贵不贵
是	是	便宜



什么是决策树（判定树）？

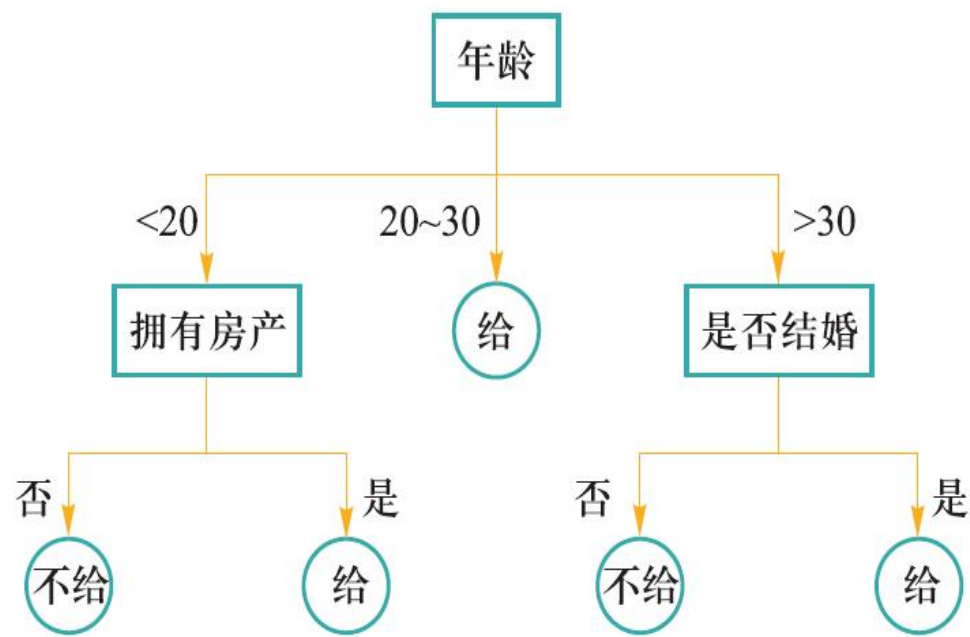
- 树结构
- 每个**内部节点**表示在一个属性上的测试
- 每个**分枝**代表一个测试输出
- 每个**树叶节点**代表类或类分布



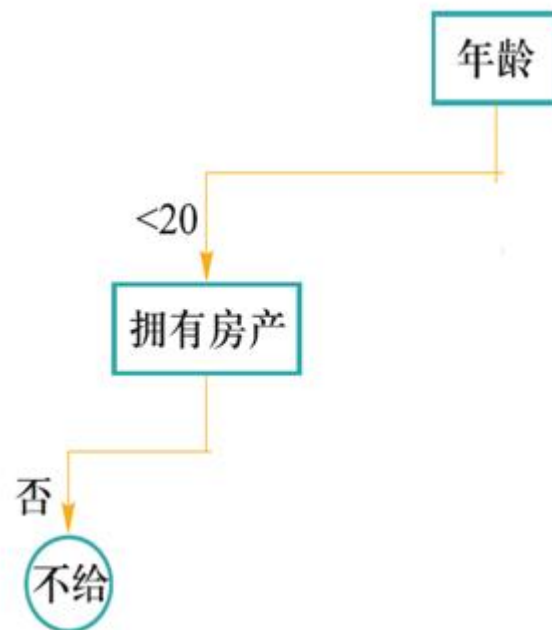
序号	年龄 / 岁	银行流水	是否结婚	是否拥有房产	是否给予贷款
1	> 30	高	否	是	否
2	> 30	高	否	否	否
3	20~30	高	否	是	是
4	< 20	中	否	是	是
5	< 20	低	否	是	是
6	< 20	低	是	否	否
7	20~30	低	是	否	是
8	> 30	中	否	是	否
9	> 30	低	是	是	是
10	< 20	中	否	是	是
11	> 30	中	是	否	是
12	20~30	中	否	否	是
13	20~30	高	是	是	是
14	< 20	中	否	否	否

决策过程：从根节点开始，测试待分类项中相应的特征属性，并按照其值选择输出分支，直到到达叶子节点，将叶子节点存放的类别作为决策结果。

例如，对于**贷款问题**，现有申请者**19岁**、无房产、未婚，是否给予贷款呢？需要根据诸如**年龄**、**有无房产**和**是否结婚**等因素做出安排或决策，。

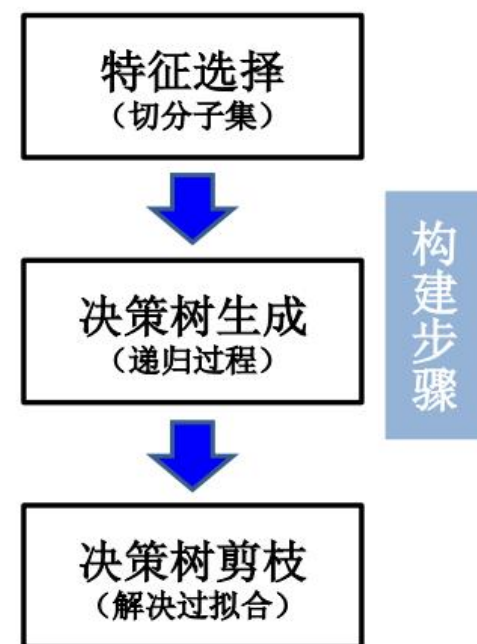


贷款问题的决策树模型



决策过程

给定训练数据，
如何构建决策树呢？



特征选择：决定用哪个特征来划分特征空间，特征选择是要选出对训练数据集具有分类能力的特征，这样可以提高决策树的学习效率。

信息熵 (Entropy)：衡量数据集的混乱程度，表示数据集 D 中类别分布的不确定性，熵越小，数据越纯净（类别越集中）；熵越大，数据越混乱。

信息增益 (Information Gain) = 信息熵(前) - 信息熵(后，也称条件熵)

$|D_i|$: 第 i 类样本数
 $|D|$: 总样本数

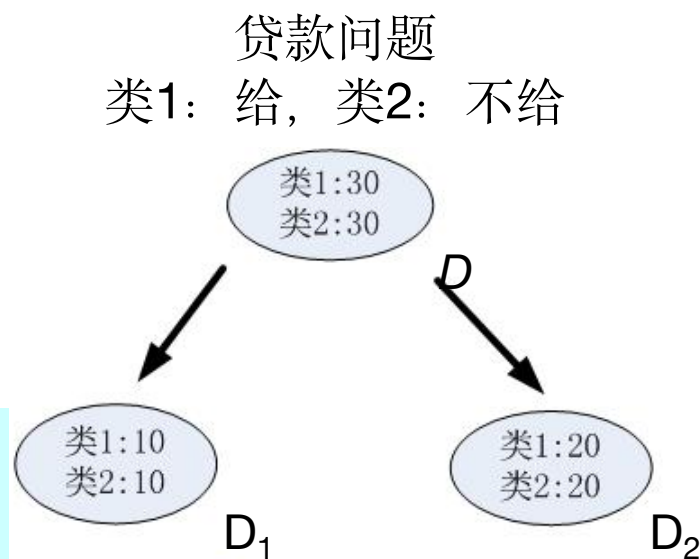
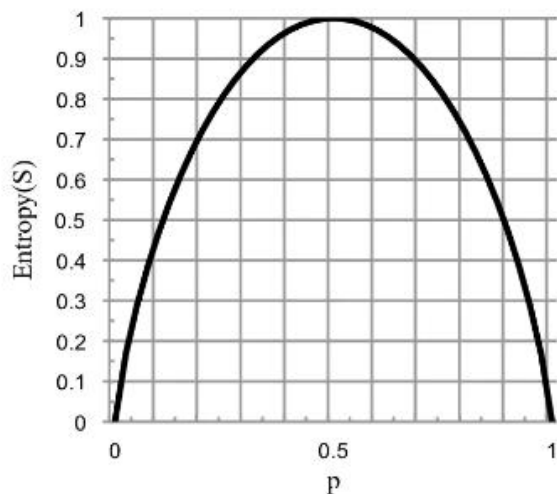
假定当前数据集 D 中有 K 类，其中第 i 类样本占比为 p_i ($p_i = \frac{|D_i|}{|D|}$)，则信息熵的计算公式：

$$E(D) = - \sum_{i=1}^K p_i \log_2 p_i$$

$$E(D) = -\frac{30}{60} \log_2 \frac{30}{60} - \frac{30}{60} \log_2 \frac{30}{60} = 1$$

二分类熵最大为1：类别间没有明显差异，正反例各占50%，分类效果最差的状态

熵最小为0：完全分类的状态，所有样本都是同一类别



一组标签 $y = \{y_i\}_{i=1}^K$

$$\text{信息熵 } E(D) = - \sum_{i=1}^K p_i \log_2 p_i$$

$$\text{信息增益 } Gain = E(D) - E(D, A)$$

$$= E(D) - \sum_{j=1}^n \frac{|D_j|}{|D|} E(D_j)$$

经验熵

- 二分类类别均分的时候熵最大
- 熵越小，混乱程度越低，纯度越高
- 信息增益越大，划分后复杂度减小得越多，分类效果越明显

条件熵 = 权重 * 经验熵，样本数越多的分支结点的影响越大

$$E(D) = -\frac{30}{60} \log_2 \frac{30}{60} - \frac{30}{60} \log_2 \frac{30}{60} = 1$$

$$E(D_1) = E(D_2) = 1$$

$$\text{信息增益: } Gain_A = E(D) - E(D, A) = E(D) - \frac{20}{60} E(D_1) - \frac{40}{60} E(D_2) = 0$$

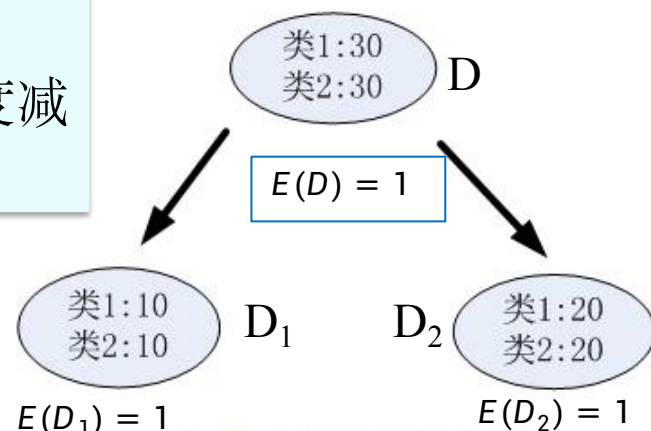
$$E(D'_1) = -\frac{28}{33} \log_2 \frac{28}{33} - \frac{5}{33} \log_2 \frac{5}{33} \approx 0.185$$

$$E(D'_2) = -\frac{2}{27} \log_2 \frac{2}{27} - \frac{25}{27} \log_2 \frac{25}{27} \approx 0.115$$

$$\text{信息增益: } Gain_B = E(D) - E(D, B) = E(D) - \frac{33}{60} E(D'_1) - \frac{27}{60} E(D'_2) \approx 0.847$$

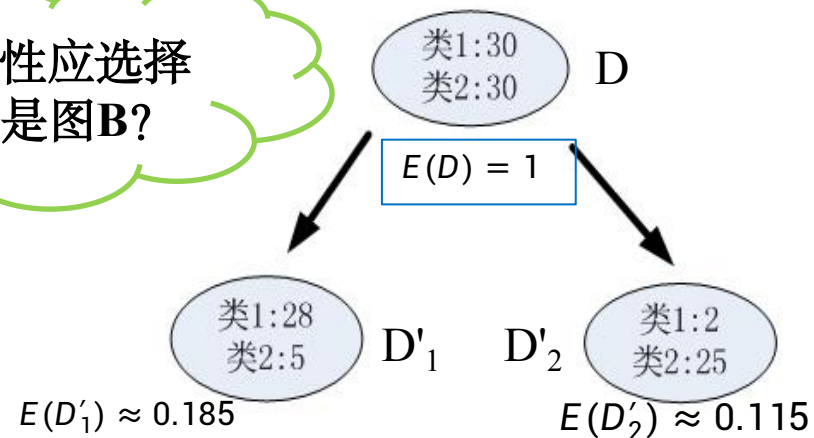
贷款问题

类1: 给, 类2: 不给



图A 按属性1进行分裂
性别

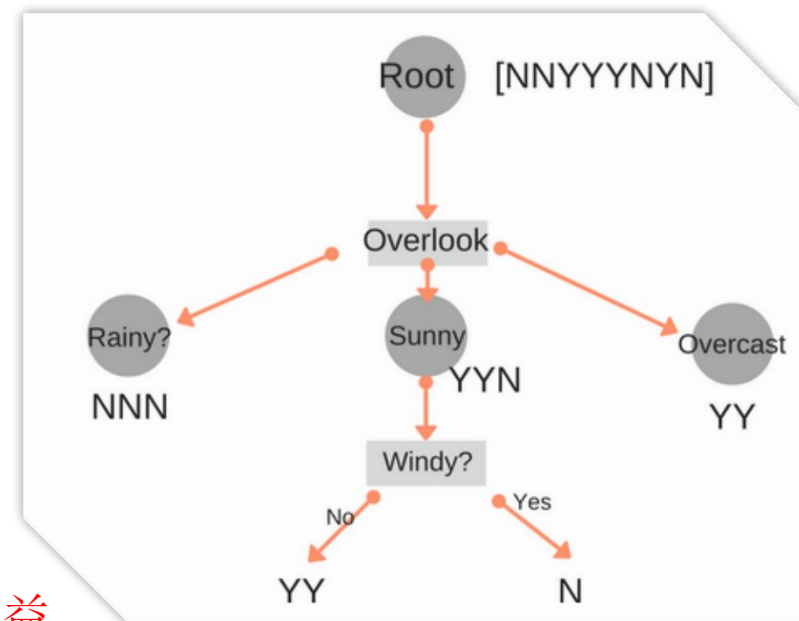
特征属性应选择
图A还是图B?



图B 按属性2进行分裂
年龄 > 30

训练数据

Predictors				Decision
Outlook	Temperature	Humidity	Windy	Play Golf?
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Overcast	Cool	Normal	TRUE	Yes
Rainy	Mild	High	FALSE	No



学到的决策树

在学习过程中，每次节点分裂（通过选择某个特征）都以**最大化信息增益**为准则（贪婪思想、局部优化）

$$E(\text{Root}) = -\frac{4}{8}\log_2\frac{4}{8} - \frac{4}{8}\log_2\frac{4}{8} = 1$$

$$E(\text{Rain}) = -\frac{3}{3}\log_2\frac{3}{3} - \frac{0}{3}\log_2\frac{0}{3} = 0$$

$$E(\text{Sunny}) = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.2442$$

$$E(\text{Overcast}) = -\frac{2}{2}\log_2\frac{2}{2} - \frac{0}{2}\log_2\frac{0}{2} = 0$$

$$\begin{aligned} \text{Gain}_{\text{Outlook}} &= E(\text{Root}) - E(\text{Root}, \text{Outlook}) \\ &= E(\text{Root}) - \frac{3}{8}E(\text{Rain}) - \frac{3}{8}E(\text{Sunny}) - \frac{2}{8}E(\text{Overcast}) \approx 0.908 \end{aligned}$$

$$\begin{aligned} \text{Gain}_{\text{Temperature}} &= E(\text{Root}) - E(\text{Root}, \text{Temperature}) \\ &= E(\text{Root}) - \frac{3}{8}E(\text{Hot}) - \frac{2}{8}E(\text{Mild}) - \frac{3}{8}E(\text{Cool}) \approx 0.543 \end{aligned}$$

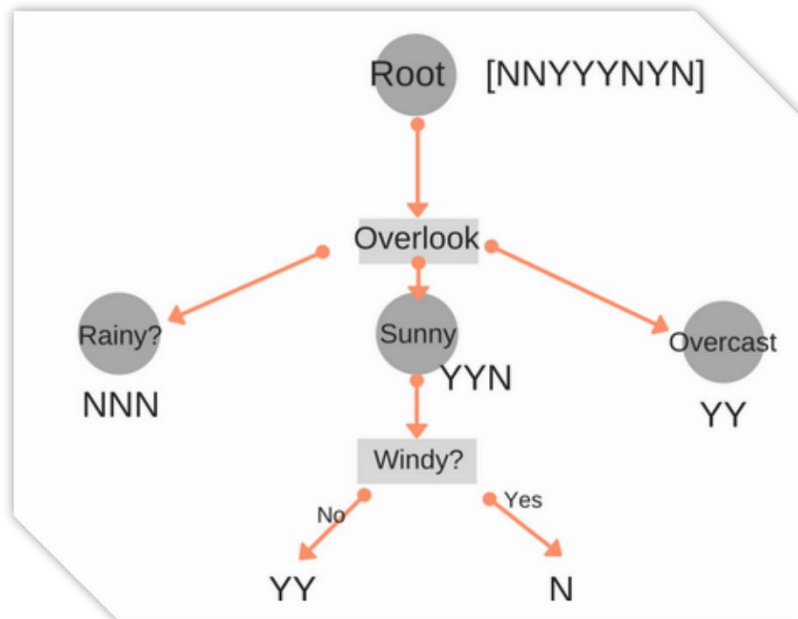
$$\text{Gain}_{\text{Humidity}} = E(\text{Root}) - \frac{5}{8}E(\text{High}) - \frac{3}{8}E(\text{Normal}) \approx 0.714$$

$$\text{Gain}_{\text{Windy}} = E(\text{Root}) - \frac{5}{8}E(\text{FALSE}) - \frac{3}{8}E(\text{TRUE}) \approx 0.714$$

ID3 算法

其大致步骤为：

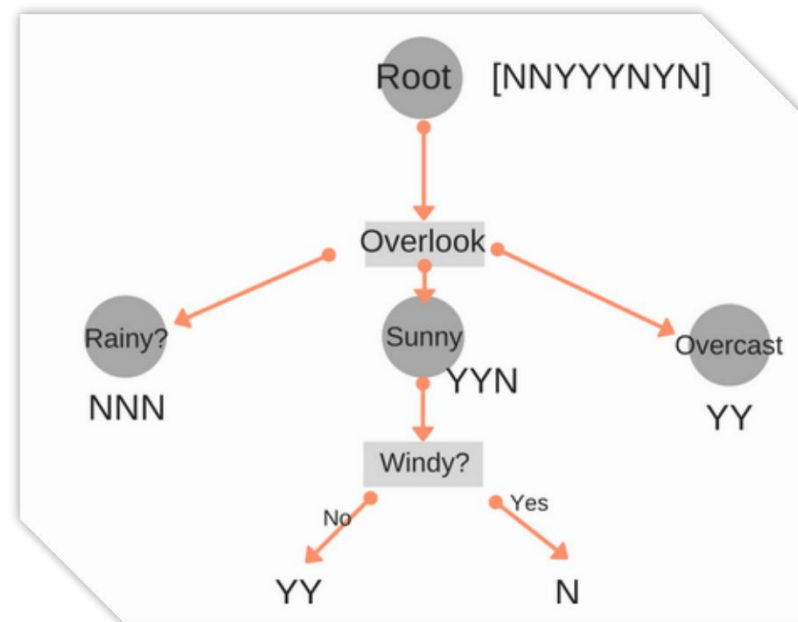
1. 初始化特征集合和数据集合；
2. 计算数据集合信息熵和所有特征的条件熵，选择信息增益最大的特征作为当前决策节点；
3. 更新数据集合和特征集合（删除上一步使用的特征，并按照特征值来划分不同分支的数据集合）；
4. 重复 2、3 两步，若子集值包含单一特征，则为分支叶子节点。



训练数据

Predictors				Decision
Outlook	Temperature	Humidity	Windy	Play Golf?
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Overcast	Cool	Normal	TRUE	Yes
Rainy	Mild	High	FALSE	No

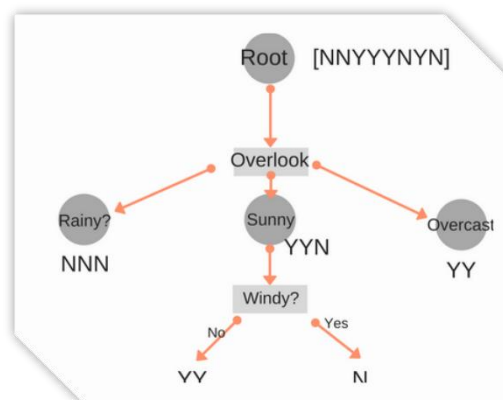
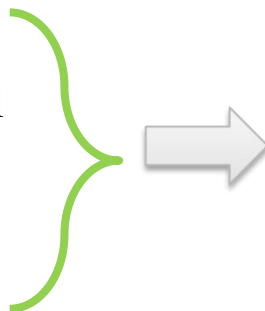
在学习过程中，每次划分（通过选择某个特征）都以**最大化信息增益**为准则（贪婪思想、局部优化）



学到的决策树

观测量或特征

Outlook : Rainy
Temperature: Cool
Humidity: High
Windy: False



学到的决策树

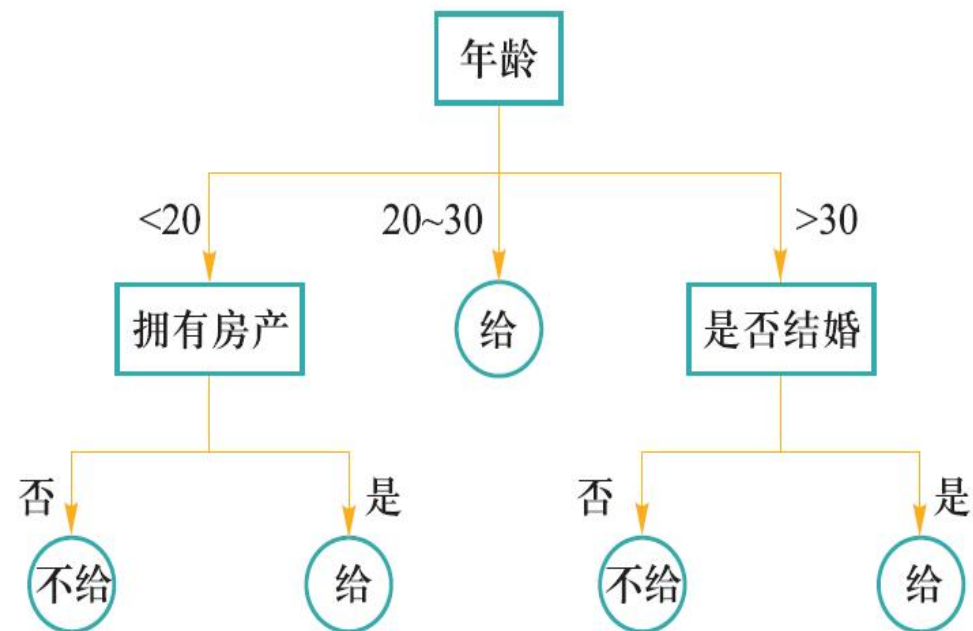
分类（预测）

应该去打高尔夫吗？

答案是**NO**

训练数据

序号	年龄 / 岁	银行流水	是否结婚	是否拥有房产	是否给予贷款
1	> 30	高	否	是	否
2	> 30	高	否	否	否
3	20~30	高	否	是	是
4	< 20	中	否	是	是
5	< 20	低	否	是	是
6	< 20	低	是	否	否
7	20~30	低	是	否	是
8	> 30	中	否	是	否
9	> 30	低	是	是	是
10	< 20	中	否	是	是
11	> 30	中	是	否	是
12	20~30	中	否	否	是
13	20~30	高	是	是	是
14	< 20	中	否	否	否

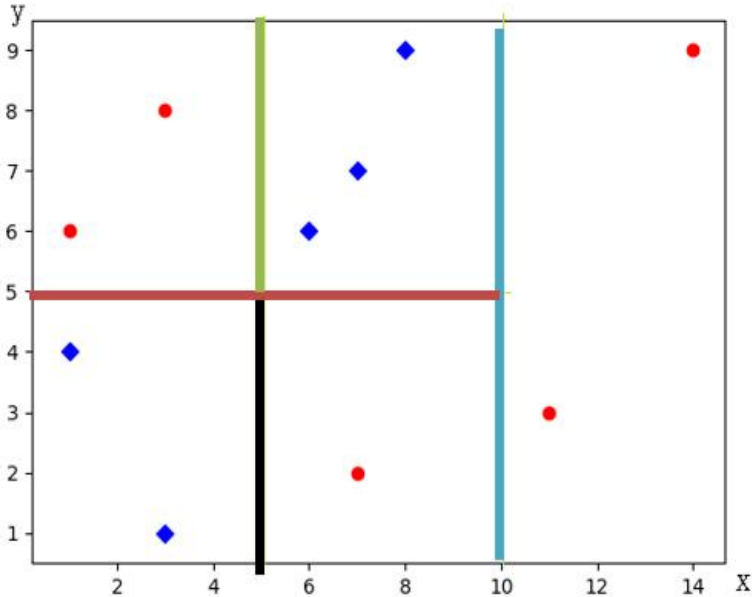
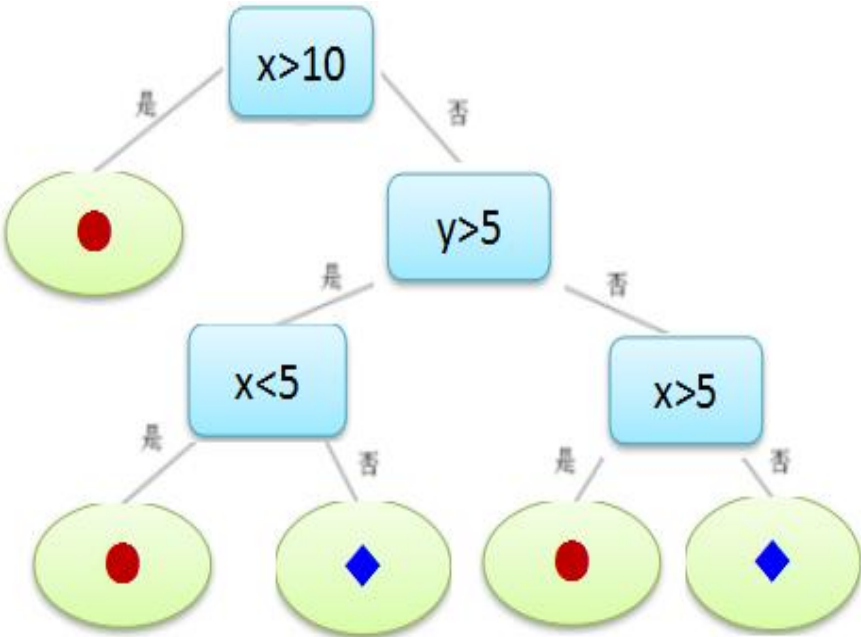


决策树

(7,11)是哪一类?

坐标	(1,4)	(1,6)	(3,1)	(3,8)	(6,6)	(7,2)	(7,7)	(8,9)	(11,3)	(14,9)
分类	◆	●	◆	●	◆	●	◆	◆	●	●

- A ◆
- B ●
- C 不知道



决策树对应二维空间的分割结果
(非线性)

提交

(7,11)是哪一类?

坐标	(1,4)	(1,6)	(3,1)	(3,8)	(6,6)	(7,2)	(7,7)	(8,9)	(11,3)	(14,9)
分类	◆	●	◆	●	◆	●	◆	◆	●	●

A

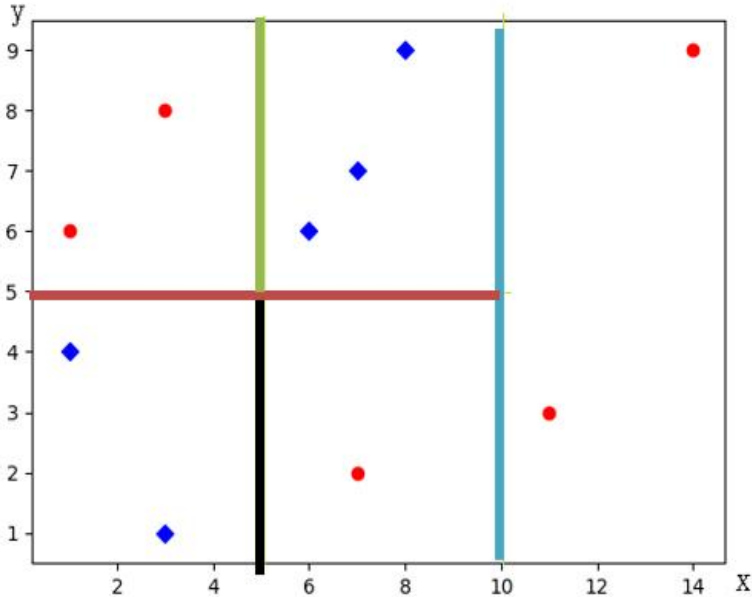
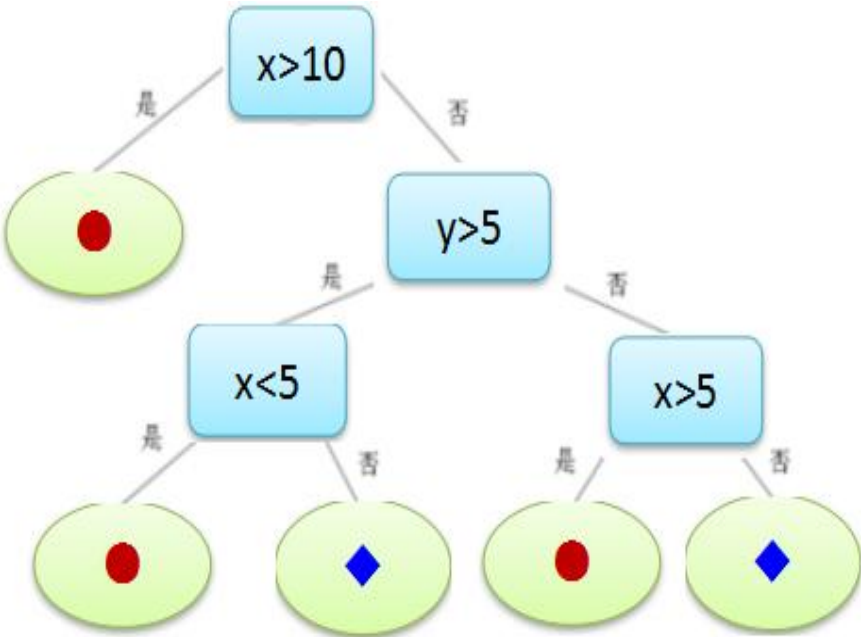


B



C

不知道



决策树对应二维空间的分割结果
(非线性)

提交

- 决策树的生成和使用总共包含三个阶段

- 决策树构建

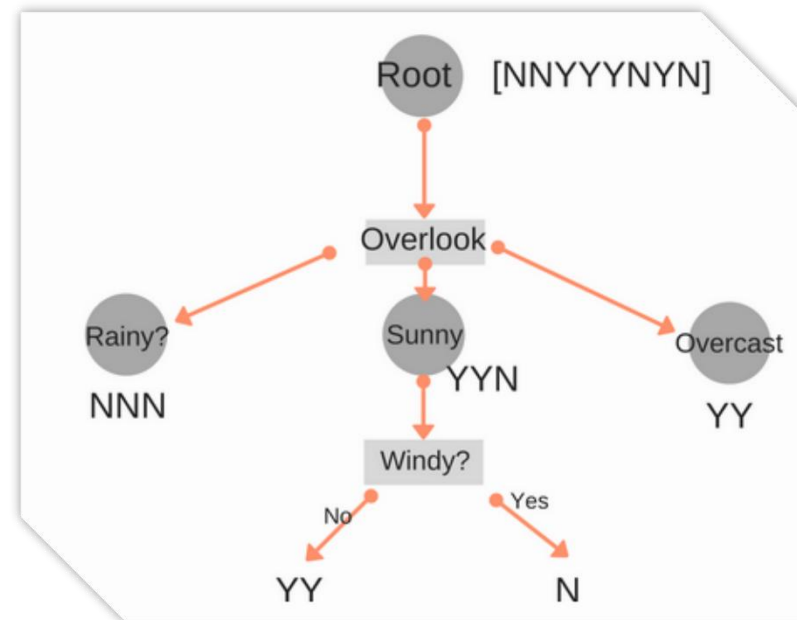
- 开始时，所有的训练样本都在根节点
- 递归地通过选定的属性来划分样本

- 决策树的使用

- 对未知样本进行分类
- 通过将样本的属性值与决策树相比较

- 决策树剪枝

- 许多分枝反映的是训练数据中的噪声和孤立点
- 决策树剪枝试图检测和剪去这种分枝



在生成树的过程中，如果没有**剪枝**（pruning）操作，就会生成一个对训练集完全拟合的决策树，但这是对测试集非常不友好的，泛化能力不行。因此，需要减掉一些枝叶，使得模型泛化能力更强。

预剪枝

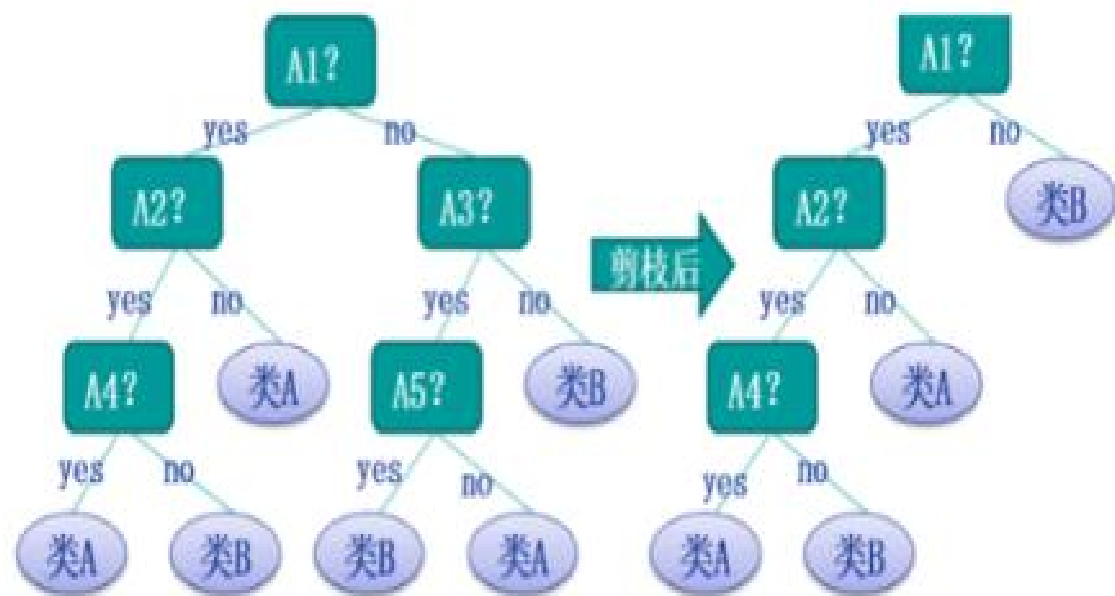
通过提前停止树的构建而对树剪枝，一旦停止，节点就是叶子，该叶子持有子集中最频繁的类。

- 定义一个高度，当决策树达到该高度时就停止生长
- 达到某个节点的实例具有相同的特征向量
- 定义一个阈值（实例个数、系统性能增益等）

后剪枝

首先构造完整的决策树，然后对那些置信度不够的结点子树用叶子结点来代替，该叶子的类标号用该结点子树中最频繁的类标记。相比于预剪枝，这种方法更常用，因为在预剪枝方法中精确地估计何时停止树增长很困难。

理想的决策树有三种：叶子节点数最少、叶子节点深度最小、叶子节点数最少且叶子节点深度最小。



决策树构建过程中的特征选择是非常重要的步骤。特征选择是决定用哪个特征来划分特征空间，特征选择是要选出对训练数据集具有分类能力的特征，这样可以提高决策树的学习效率。

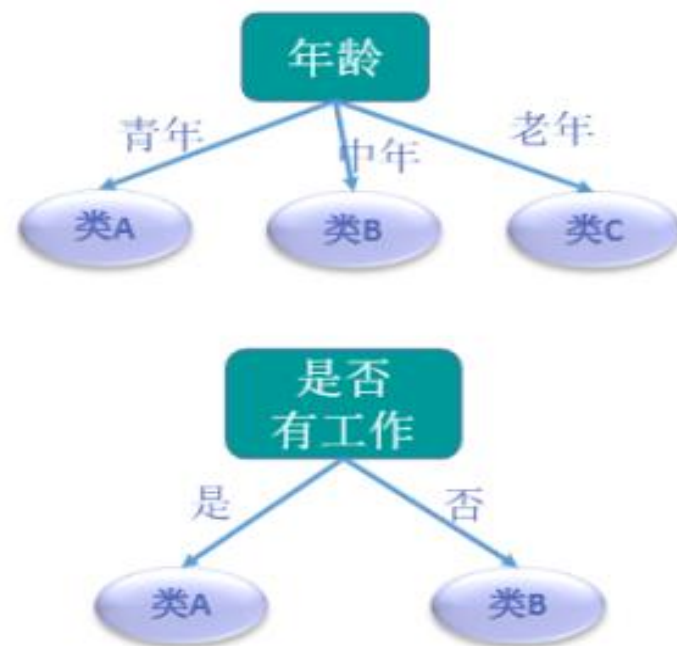
信息熵：表示随机变量的不确定性，熵越大不确定性越大。

信息增益：信息增益 = 信息熵(前) - 信息熵(后)

信息增益比：信息增益比 = 惩罚参数 * 信息增益。特征个数较多时，惩罚参数较小；特征个数较少时，惩罚参数较大。

基尼指数：表示集合的不确定性，基尼系数越大，表示不平等程度越高。

算法	支持模型	树结构	特征选择
ID3	分类	多叉树	信息增益
C4.5	分类	多叉树	信息增益比
CART	分类、回归	二叉树	基尼系数，均方差



- ◆ 使用信息增益作为选择分裂的条件有一个不可避免的缺点：倾向选择分支比较多的属性进行划分。
- ◆ CART 使用 Gini 系数作为变量的不纯度量，减少了对数运算。
- ◆ CART 采用“基于代价复杂度剪枝”方法进行剪枝，而 C4.5 采用悲观剪枝方法。

特征选择

- 决定在当前节点上使用哪个输入特征来进行节点分割数据。
- 常用选择标准：信息增益、信息增益比和基尼指数。

决策树生成

- 递归过程。
- 从根节点开始，选择最佳特征来分割数据，然后对每个子节点重复该过程，直到满足停止条件。

决策树剪枝

- 解决过拟合。
- 预剪枝：在生成过程中设置限制条件。
- 后剪枝：先生成完整的树，然后删除贡献不大的子树。

决策树核心算法：ID3算法，C4.5算法及CART算法

实际应用中，决策树常常是构建更复杂模型（如随机森林、梯度提升树）的基础。

在决策树的学习过程中，ID3算法每次划分选择特征的标准是什么？

- ☐ A 随机选择
- ☐ B 最大化信息增益
- ☐ C 最大化信息增益比
- ☐ D 最大化Gini系数

- ID3算法 适合小规模、离散特征、类别明确的分类任务
- 对于大规模数据、连续特征或高维数据，更常用 C4.5、CART（分类回归树）或基于树的集成算法（随机森林、XGBoost）。

提交

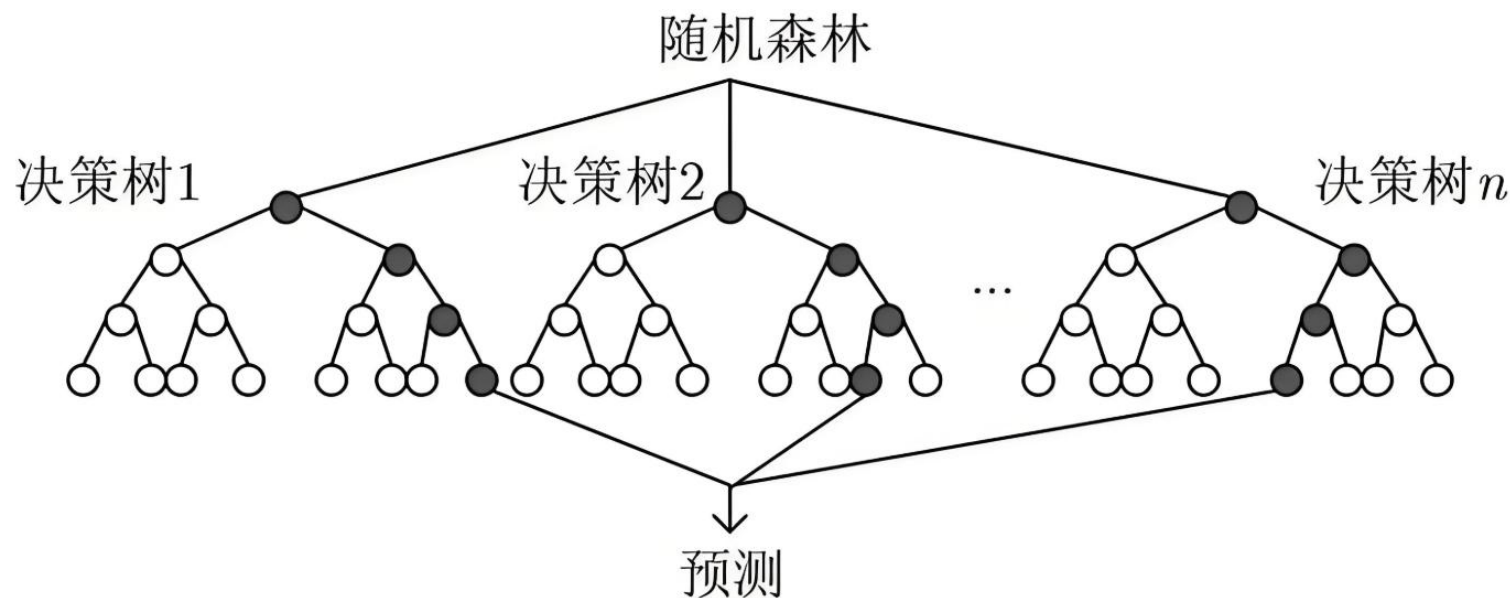
在决策树的学习过程中，ID3算法每次划分选择特征的标准是什么？

- ☐ A 随机选择
- ☒ B 最大化信息增益
- ☐ C 最大化信息增益比
- ☐ D 最大化Gini系数

- ID3算法 适合小规模、离散特征、类别明确的分类任务
- 对于大规模数据、连续特征或高维数据，更常用 C4.5、CART（分类回归树）或基于树的集成算法（随机森林、XGBoost）。

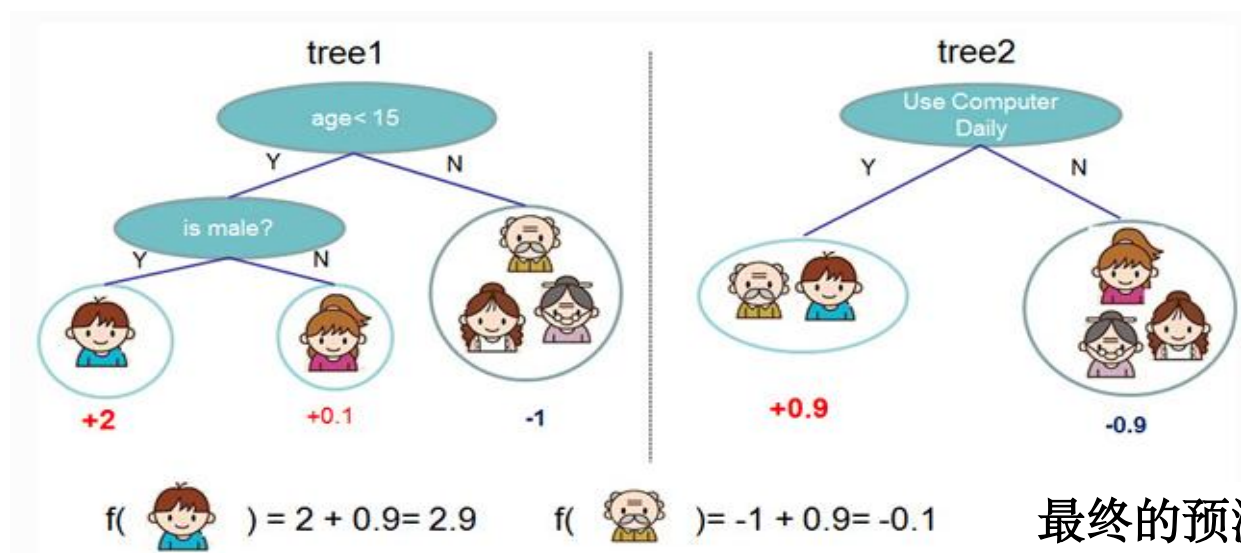
提交

- **随机森林**: 基于决策树的集成学习方法, 属于Bagging类型, 通过构建多棵决策树并让它们共同投票来决定最终的分类结果, 较好地解决单棵决策树容易过拟合和局部最优的问题。
 - **Tree bagging**: 多次**有重复随机采样**训练样本, 对每次采样训练一棵决策树, 预测结果等于所有树对不同标签进行**投票(voting)**



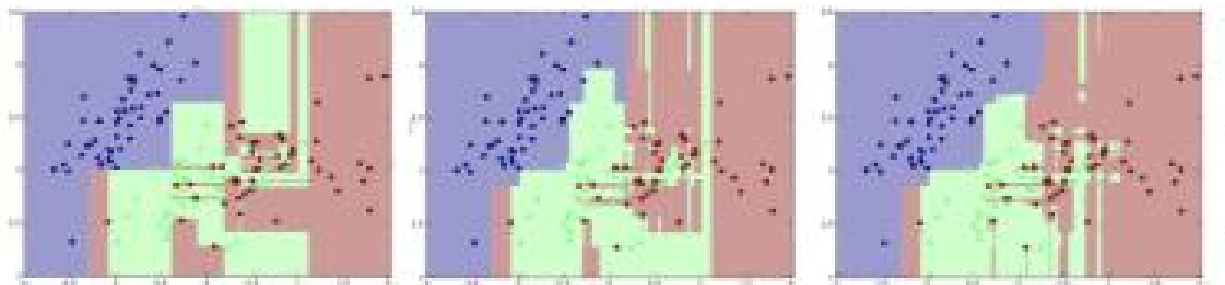
- **Tree bagging:** 多次有重复随机采样训练样本，对每次采样训练一棵决策树，预测结果等于所有树对不同标签进行投票(voting)
- **Feature bagging:** 训练决策树时，每次划分都基于一组随机特征进行（消除强特征带来的决策树关联问题）

对一个人是否喜欢电脑游戏进行分类



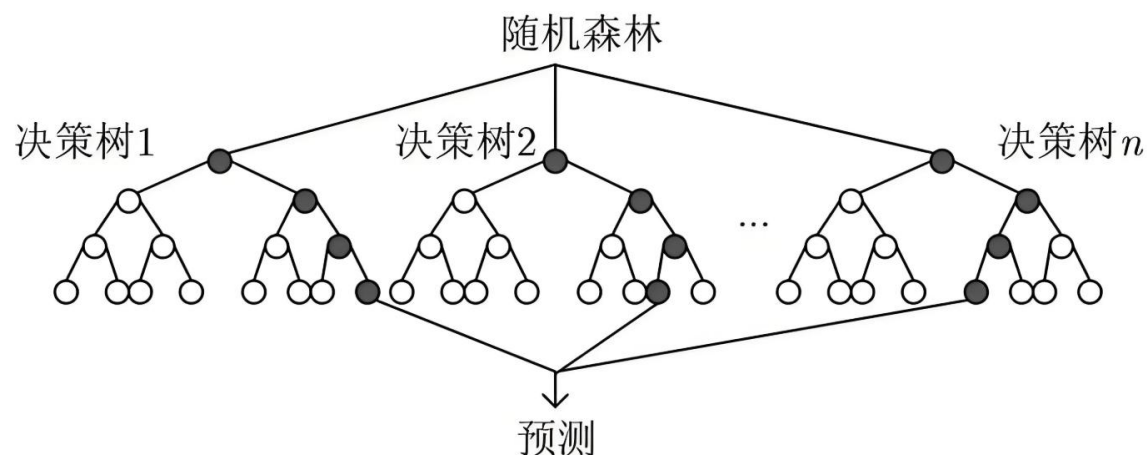
最终的预测值就是样本在每颗树中所在的叶子结点的预测值的和

回归树集成(Feature bagging)



单颗决策树

Show Me AI

5颗决策树的
随机森林25颗决策树的
随机森林

➤ 随着随机森林中决策树数量的增多，模型的泛化能力逐渐增强，决策边界越来越趋于平滑

➤ 参数调优:

1) RF划分时考虑的最大特征数 `max_features`

2) 决策树的棵数 `n_estimators > 50`

3) 决策树最大深度 `max_depth: 4~12`

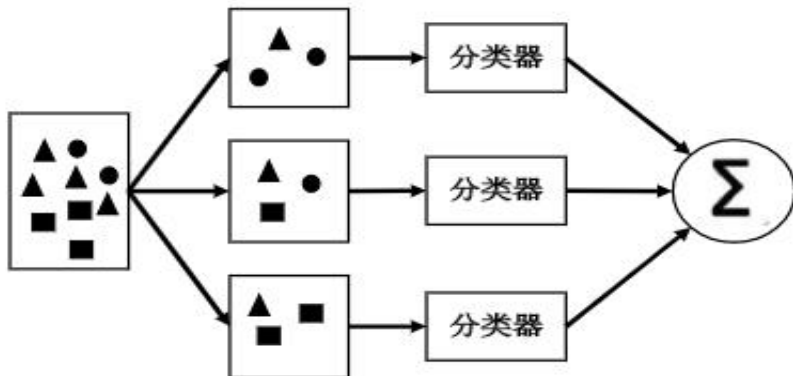
4) 内部节点再划分所需最小样本数 `min_samples_split: 16, 32, 64`等

5) 叶子节点最少样本数 `min_samples_leaf > 1`

集成学习通过将多个弱分类器集成在一起，使它们共同完成学习任务，构建一个强分类器。潜在哲学思想是“三个臭皮匠赛过诸葛亮”。



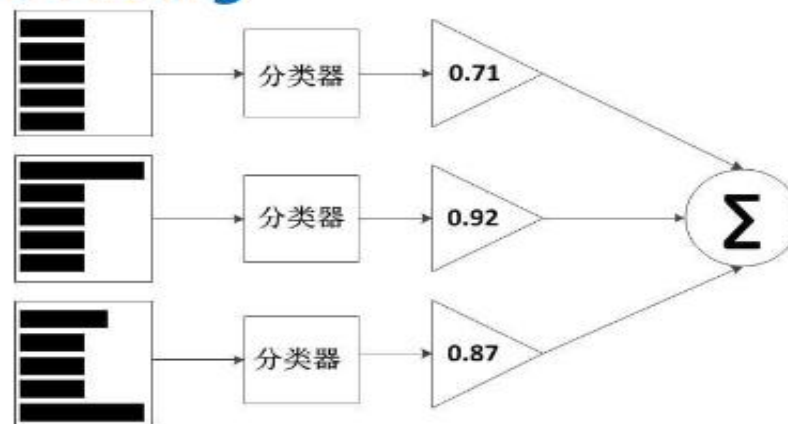
Bagging(bootstrap aggregating)



Bagging: 基于数据**随机重抽样**的分类器构建方法

- 利用bootstrap方法从整体数据集中采取有放回抽样得到N个数据集（如何采样？）
- 在每个数据集上学习出一个模型（选择什么样的弱分类器？）
- 利用N个模型的输出投票得到最后的预测结果

Boosting



Boosting (Adaptive Boosting的简称)，基于错误提升分类器性能，通过**集中关注被已有分类器分类错误的样本**，构建新分类器。

- 初始的分布应为等概分布。
- 每次循环后提高错误样本的分布概率，分错的样本在训练集中所占权重增大，使得下一次循环的基分类器能够集中力量对这些错误样本进行判断。
- 计算分类器的权重，识别率越高的基分类器权重越高，识别率越低的基分类器权重越低。



- ID3是主流决策树算法之一，基于信息增益进行特征选择和树的生长。
- C4.5是ID3算法的改进，基于信息增益率来选择最优属性。
- CART是可用于分类与回归任务的二叉决策树，广泛使用。
- RF (Random Forest, 随机森林) 是把决策树并行集成的组合算法。
- GBDT、XGBoost/LightGBM: boosting系列模型，可以基于树模型做串行集成。

- 决策树引入及使用

Scikit-learn 实现了 CART (Classification and Regression Tree) 决策树算法, 并未直接提供 ID3、C4.5、C5.0 的官方实现

```
from sklearn import tree, model_selection, metrics
```

#类似前面的分类与预测算法, 数据集同样要进行训练集和测试集的划分

```
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.2, random_state=42)
```

#根据训练数据训练决策树模型

```
dc_tree = tree.DecisionTreeClassifier(criterion = 'entropy' , min_samples_leaf = 5)
```

```
dc_tree.fit(x_train, y_train)
```

criterion: 'gini' 或者 'entropy' (基尼系数/信息增益)

max_depth: 最大深度, 默认为None, 自动生长

min_samples_split: 分裂内部节点的最小样本数, 默认为2

min_samples_leaf: 叶子结点的最小样本数, 默认为1

random_state: 设置随机种子, 保证结果可复现

- 决策树引入及使用

#根据测试数据进行预测并计算准确率

```
y_pred=tree.predict(x_test)
```

```
accuracy=tree.score(y_test, y_pred)
```

#计算模型性能——准确率、召回率、F1分数

```
print("分类报告: \n", tree.classification_report(y_test, y_pred))
```

#输出特征重要性

```
feature_importances=tree.feature_importances_
```

```
feature_names=data.feature_names
```

```
for name, importance in zip(feature_names, feature_importances):
```

```
    print(f"{name}:{importance:.4f}")
```

- 决策树引入及使用

#可视化整棵树，更直观的查看决策树分裂情况

```
from sklearn.tree import plot_tree
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 8))
```

```
plot_tree(tree, feature_names=data.feature_names, class_names= data.target_names,  
filled=True)
```

```
plt.show()
```

#以文本格式导出决策树规则

```
from sklearn.tree import export_text
```

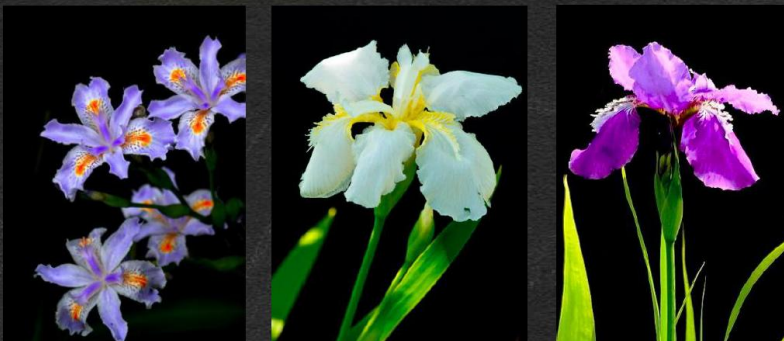
```
tree_rules = export_text(tree, feature_names=data.feature_names)
```

```
print(tree_rules)
```

- 决策树引入及使用
 - 除了决策树（单棵决策树，易解释，但是容易过拟合，比较适合小数据）之后，还有一个更常用的随机森林（多棵决策树组合Bagging，减少过拟合，泛化能力更强）

```
from sklearn.ensemble import RandomForestClassifier  
  
rf = RandomForestClassifier(n_estimators=100, random_state=42)  
  
rf.fit(X_train, y_train)  
  
print("随机森林准确率:", rf.score(X_test, y_test))
```
 - 还有诸如ExtraTreesRegressor等，大家可以自行学习

下列鸢尾花分别属于哪一类：setosa、versicolor、virginica



样本

Samples
/ (instances, observations)

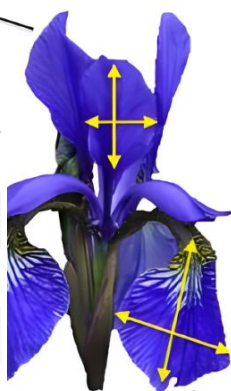
	sepal length	sepal width	petal length	petal width	target	label
0	5.1	3.5	1.4	0.2	Iris-setosa	0
1	4.9	3.0	1.4	0.2	Iris-setosa	0
2	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5.0	3.6	1.4	0.2	Iris-setosa	0

Features
(attributes, measurements, dimensions)

特征

花瓣

Petal



Class labels
(targets)

类别标签

Sepal
花萼

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from matplotlib import pyplot as plt
data = pd.read_csv('iris_data.csv' )
X = data.drop(['target','label'], axis =1)
y = data.loc[:, 'label']

#构建决策树并训练
dc_tree =
DecisionTreeClassifier(criterion='entropy',min_samples_leaf=5)
dc_tree.fit(X, y)

# 可视化决策树
fig = plt.figure(figsize=(20,20))
plot_tree(dc_tree, filled =True, feature_names =['sepal
length','sepal width','petal length','petal width'],
class_names=['setosa','versicolor','virginica'])
```

下列鸢尾花分别属于哪一类：setosa、versicolor、virginica



样本

花瓣

Petal

 Samples
(instances, observations)

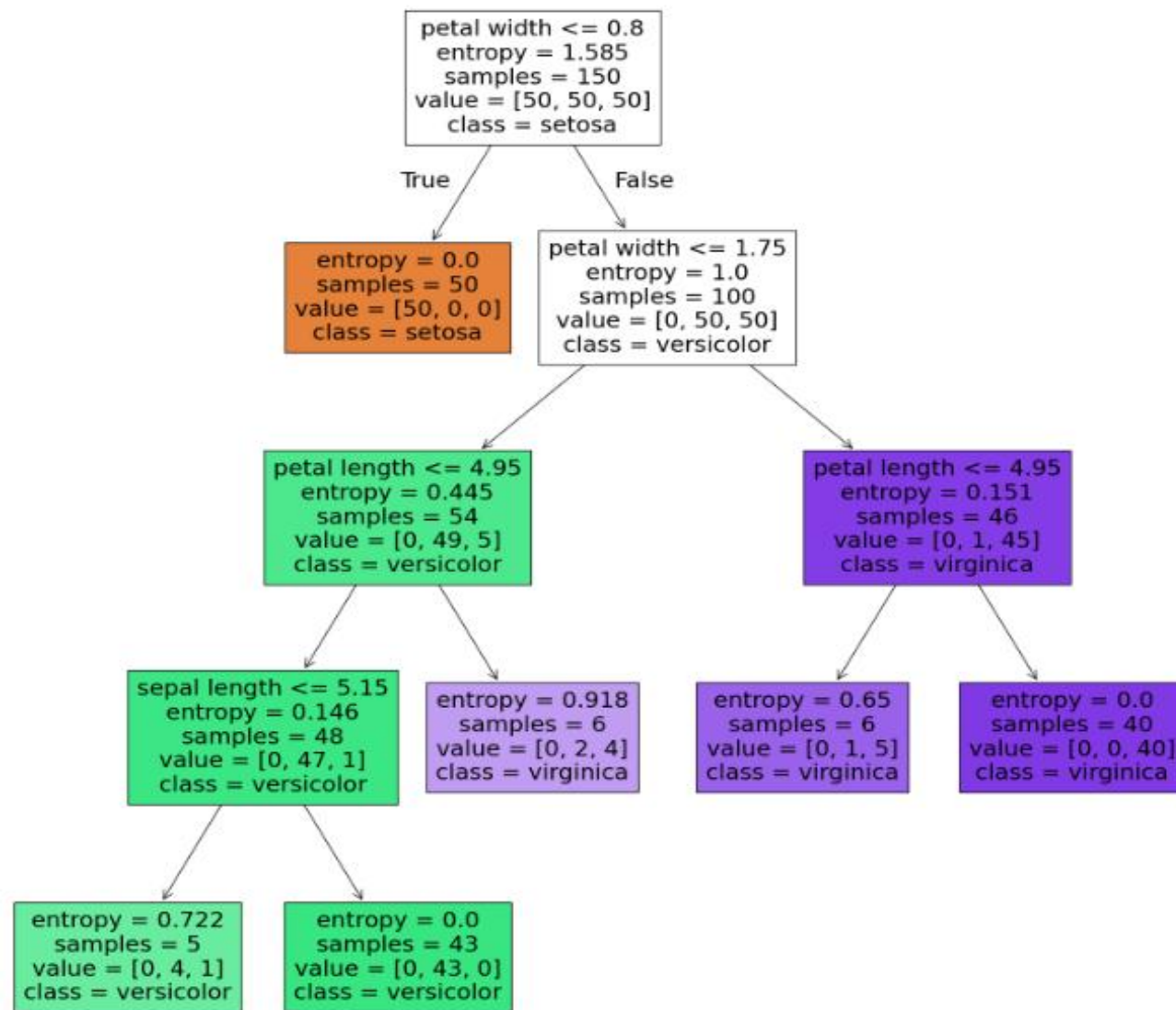
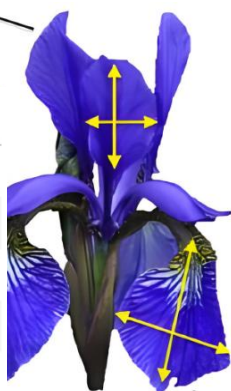
	sepal length	sepal width	petal length	petal width	target	label
0	5.1	3.5	1.4	0.2	Iris-setosa	0
1	4.9	3.0	1.4	0.2	Iris-setosa	0
2	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5.0	3.6	1.4	0.2	Iris-setosa	0

 Features
(attributes, measurements, dimensions)

特征

Class labels
(targets)

类别标签

 Sepal
花萼


决策树输出结果展示



- (1) **ID3**算法中，决策树如何选择特征进行节点分裂？
- (2) 请给出决策树防止过拟合的策略，至少给出两个策略。
- (3) 请简述集成学习的优点。